Lógica y Representación 1

Funciones - Introducción

Ingeniería de Sistemas

Universidad de Antioquia

Método de Polya - Resumen





PROBLEMA

ALGORITMO



1. Entender el problema
2. Diseñar un plan
3. Implementar el plan
4. Revisar el plan

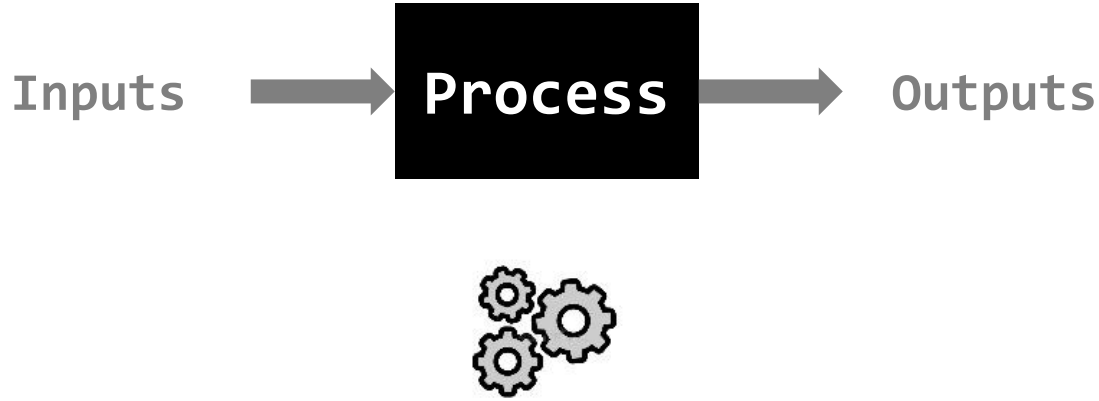
Entrada, procesamiento y salida

Los programas de computadora realizan estos tres pasos:

1. Se recibe la **entrada**.
2. Se realiza un **proceso** sobre la entrada.
3. Se genera la **salida**

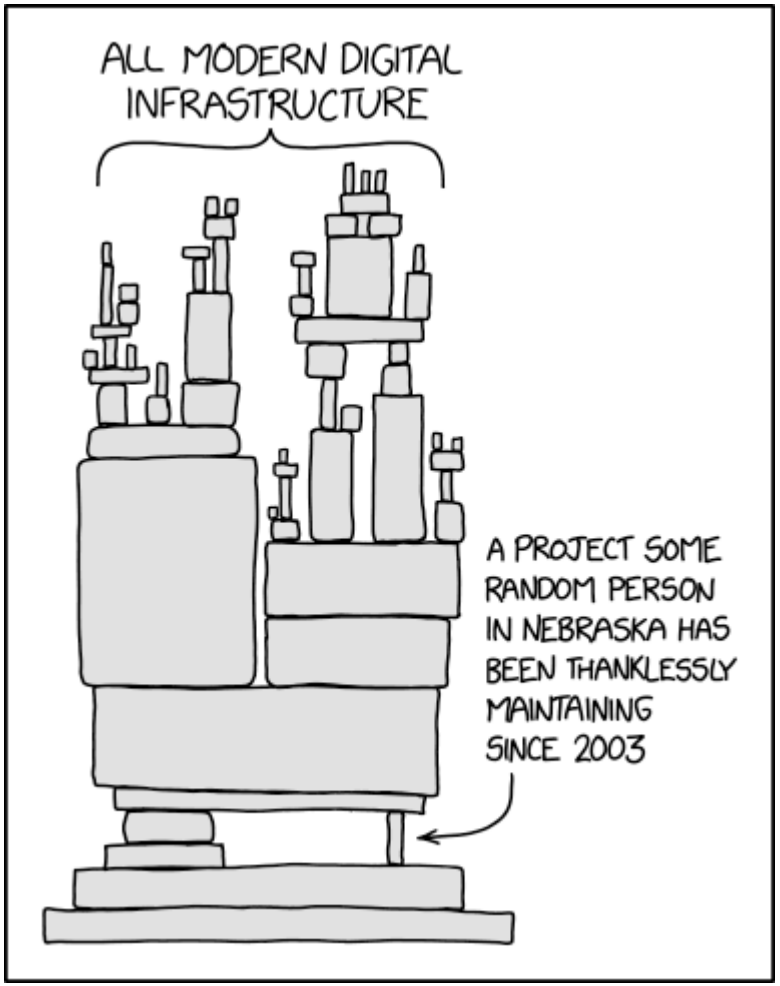






Introducción a las funciones

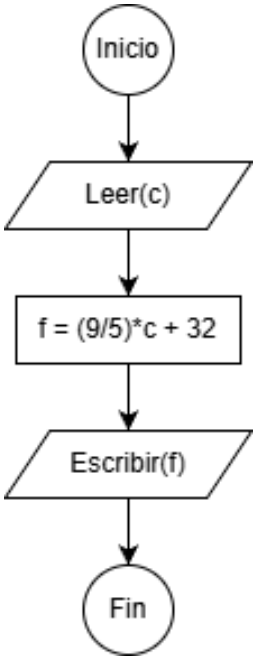
```
int getRandomNumber()
{
    return 4;  // chosen by fair dice roll.
               // guaranteed to be random.
}
```



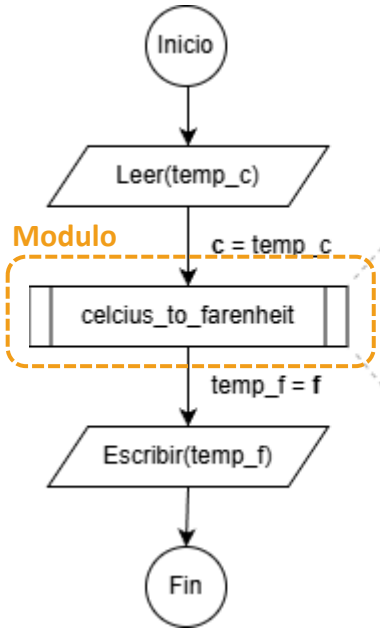
Función

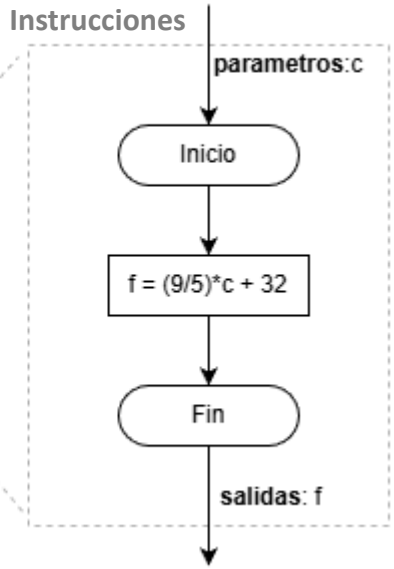
Función:

- Porción de código que realiza una tarea especifica.
- También se conoce como: subrutina, procedimiento, función o método.



Modularizar






```
c = float(input("Ingrese los °C: "))
f = (9.0/5)*c + 32
print(c,"°C =",f, "°F")
```

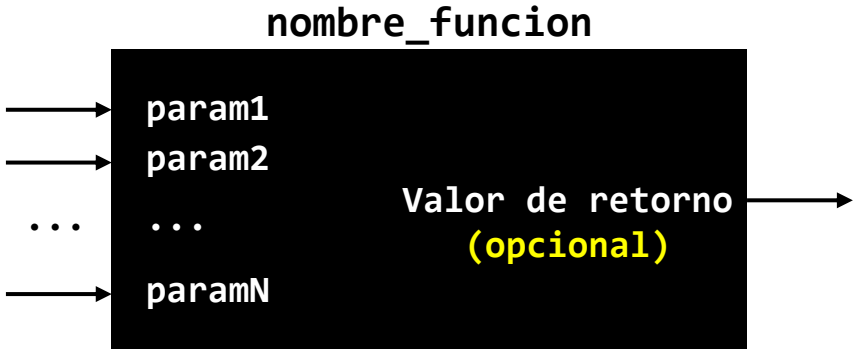
Link simulación

Funciones en Python - Definición

- **Sintaxis**: Una función se define mediante la palabra clave **def** seguido del nombre del nombre y los parámetros entre paréntesis:

```
def nombre_funcion(param1, param2 ,... , paramN):
    cuerpo de la funcion
```

- **Donde**:
  - **Nombre de la función**: Identifica la función; con este se llama (invoca).
  - **Parámetros**: Lista de entradas de la función (puede estar vacía).
  - **Cuerpo de la función**: Conjunto de instrucciones que realizan la tarea (van indentadas).
- **Importante**: Definir una función no la ejecuta: solo se ejecuta cuando se la llama.
- **Diagrama de caja negra**: Resume la interfaz de la función (entradas/salidas) sin entrar en los detalles internos.



Funciones en Python - Invocación

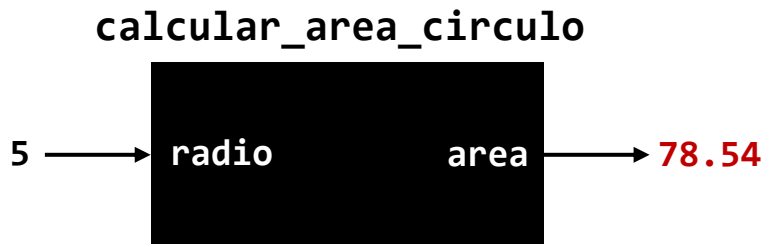
- **Invocar (llamar) una función**: escribir su nombre con los argumentos entre paréntesis

```
[variable =] nombre_funcion(arg1, arg2 ,... , argN)
```

- **Donde**:
  - **Argumentos**: los valores que se envían, y se asocian a los parámetros **por posición**. Pueden ser números, variables o expresiones.
  - Si la función **devuelve** un valor, se guarda en una variable o se usa directamente. Si no devuelve nada, se la llama sola.
- **Ejemplo**: Empleando la función `calcular_area_circulo()`, calcular el área de un circulo de radio 5 y llevar el resultado a una variable llamada área.

```
area = calcular_area_circulo(5)
```

Link simulación



Probar una función

- **Prueba unitaria**: verificar una función de forma aislada, con varios casos de uso.
- Cada **caso de prueba** compara el resultado **esperado** con el **obtenido**.
- **Integración**: proceso de combinar las funciones ya desarrolladas en el programa completo (menú, lectura de datos y ciclo principal).
- **¿Por qué es importante probar antes de integrar?**
  - Si una función falla de forma aislada, el error se encuentra en ella.
  - Si falla dentro del programa completo, el origen puede estar en la función, en la lectura de datos o en el menú, y localizarlo resulta más costoso.
  - Una función verificada puede reutilizarse con confianza.

Tipo de caso	Qué prueba	Ejemplo
Típico	Uso habitual	`calcular_area_rectangulo(4, 3) → 12`
Límite	Valores extremos del rango válido	`calcular_area_rectangulo(0, 5) → 0`
Especial	Valores atípicos o inválidos	`calcular_area_rectangulo(-4, 3) → -12`, que la función no rechaza

Pruebas con `assert`

- **Sintaxis**: Una verificación se escribe con la palabra clave **`assert`**, seguida de una condición y, opcionalmente, un mensaje:

```
assert condición [, "Mensaje opcional"]
```

- **Donde**:
  - **Condición**: expresión que debe ser verdadera, normalmente la comparación **`llamada == resultado_esperado`**.
  - **Mensaje (opcional)**: texto que se muestra si la condición es falsa.
- **Si la condición se cumple**, el programa continúa sin mostrar nada.
- **Si no se cumple**, Python lanza **`AssertionError`** e indica la línea que falló o muestra el mensaje (en caso de que este se haya definido).

```
# Sin mensaje
assert calcular_area_rectangulo(4, 3) == 12

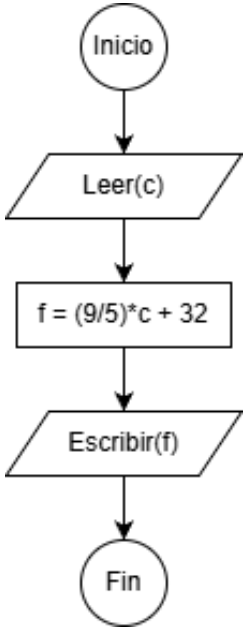
# Con mensaje
assert calcular_area_rectangulo(4, 3) == 12, "El área de 4 x 3 debe ser 12"
```

Link simulación

Salida en pantalla (Vacía – Todo salió bien)

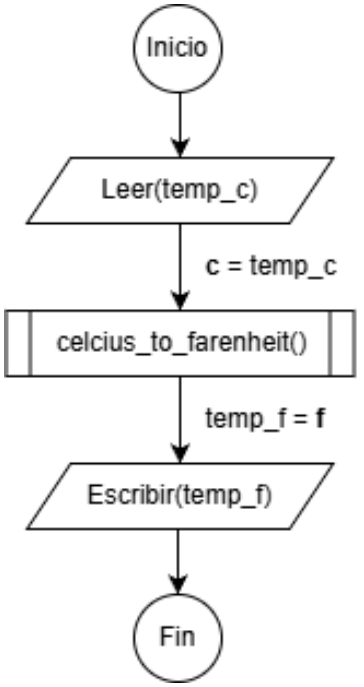
Comparación

- A continuación, se muestra la comparación entre trabajar un programa **sin módulos** y **con módulos**.




```
temp_c = float(input("Ingrese los °C: "))
temp_f = (9 / 5) * temp_c + 32
print(f"{temp_c} °C = {temp_f} °F")
```

Link simulación




```
def celsius_a_fahrenheit(c):
    f = (9 / 5) * c + 32
    return f


temp_c = float(input("Ingrese los °C: "))
temp_f = celsius_a_fahrenheit(temp_c)

print(f"{temp_c} °C = {temp_f} °F")
```

Link simulación

Ejemplo – Conversión de temperatura

Una estación meteorológica necesita un programa que convierta temperaturas medidas en grados Celsius (°C) a otras dos escalas: grados Fahrenheit (°F) y Kelvin (K). Al iniciar, el programa muestra un título y luego un menú con tres opciones:

```
1. °C a °F
2. °C a K
3. Salir
```

El usuario elige una opción y el programa hace lo siguiente:

- **Opción 1**: lee una temperatura en °C, la convierte a °F con la fórmula $F = (9/5)C + 32$ y muestra el resultado con dos decimales.
- **Opción 2**: lee una temperatura en °C, la convierte a Kelvin con la fórmula $K = C + 273.15$ y muestra el resultado con dos decimales.
- **Opción 3**: muestra un mensaje de despedida y termina el programa. Cualquier otro número: muestra el mensaje `ERROR: Opcion no valida` y vuelve a mostrar el menú.

Después de cada conversión, o de un error, el menú se muestra otra vez. El programa se repite hasta que el usuario elige la opción 3.

Ejemplo – Conversión de temperatura

Requisito: el programa debe estar organizado con funciones:

- Una función que muestre el menú.
- Una función por cada conversión, que reciba la temperatura en °C como parámetro y **devuelva** el valor convertido. Las funciones de conversión no leen ni imprimen nada: eso lo hace el programa principal.

Pruebe cada función de conversión por separado antes de integrarla al programa principal.

Solución: La implementación inicial buscaba aterrizar las bases de la programación modular (usando funciones). Sin embargo, aunque la solución propuesta cumplía con este requerimiento, no empleo buenas practivas, por lo que tomando como punto de partida la versión inicial (temperaturas_v1.py) vamos a ir mejorando el codigo inicial conforme iremos aplicando buenas practicas.

Ejemplo – Conversión de temperatura

Análisis y diseño de la solución

- **Entradas:**
  - La opción del menú (entero)
  - La temperatura en °C (real).

- **Salidas**:
  - La temperatura convertida con dos decimales.
  - Mensaje de error o de despedida.

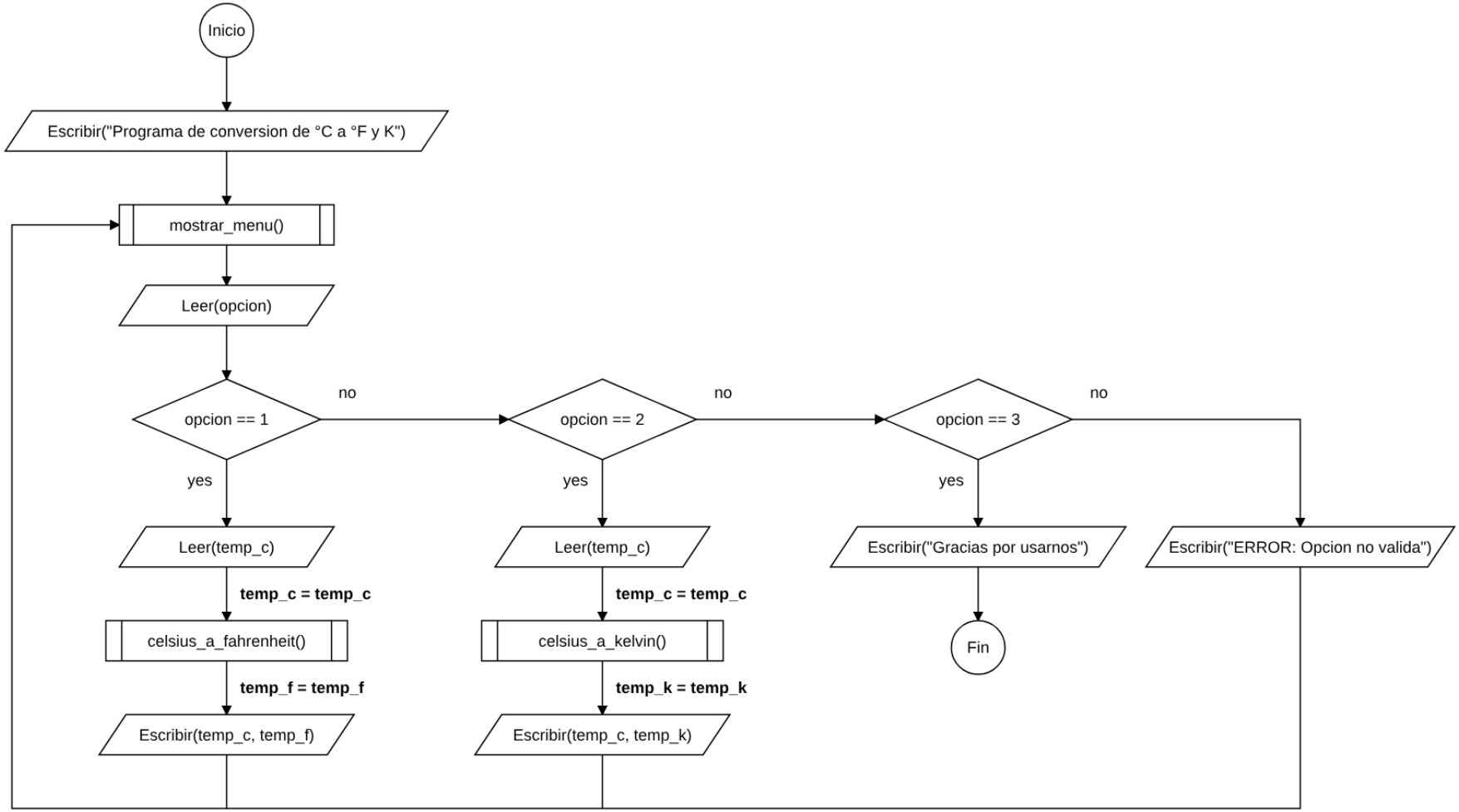
Ejemplo – Conversión de temperatura

Análisis y diseño de la solución

- **Casos de prueba:**

Opción	Entrada (°C)	Salida esperada	Tipo
1	100	100.0°C = 212.00°F	Típico (punto de ebullición)
1	0	0.0°C = 32.00°F	Típico (punto de congelación)
1	37	37.0°C = 98.60°F	Típico (temperatura corporal)
1	-40	-40.0°C = -40.00°F	Especial (las dos escalas coinciden)
2	25	25.0°C = 298.15K	Típico
2	-273.15	-273.15°C = 0.00K	Límite (cero absoluto)
7	–	ERROR: Opcion no valida	Especial (opción inexistente)
3	–	Gracias por usarnos	Salida del programa

Diagrama de flujo del programa



La implementación en Python del algoritmo anterior, constituye la primera mejora respecto a la versión inicial (temperaturas_v1.py). El codigo mejorado y asociado a este diagrama de flujo, se encuentra en el script (temperaturas_v2.py)

Buenas prácticas

Antes y después: el mismo programa, mejor escrito.

temperaturas_v1.py (hecho en clase)	**temperaturas._v2.py (mejorado)**
`def menu():` `  # Code...`	`def mostrar_menu():` `  # Code...`

`def c_to_f(c):`
`  f = (9/5)*c + 32`
`  return f`
	`def celsius_a_fahrenheit(temp_c):`
	`  temp_f = (9/5)*temp_c + 32`
	`  return temp_f`

`Tc = float(input(...))` `Tf = c_to_f(Tc)`	`temp_c = float(input(...))` `temp_f = celsius_a_fahrenheit(temp_c)`
`Pruebas con print, comentadas`	`Pruebas con assert`

Reglas:

1. **El nombre de una función empieza con un verbo y dice qué hace**: **`mostrar_menu`**, no **`menu`**.
2. **Se usan nombres completos, no abreviaturas crípticas**: **`celsius_a_fahrenheit`**, no **`c_to_f`**.
3. **Las variables van en minúscula con guion bajo (snake_case)**: **`temp_c`**, no **`Tc`**.
4. **Las pruebas se escriben con `assert`, para que se revisen solas y no a ojo**.

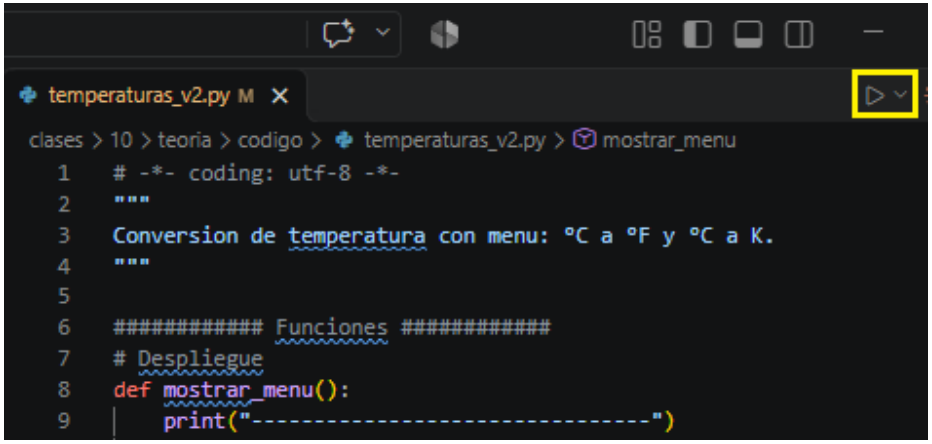
Idea clave: los dos programas hacen exactamente lo mismo. El segundo se entiende sin necesidad de explicarlo.

¿Cómo ejecutar un programa?

Idea clave: hay varias formas de ejecutar un programa, pero todas hacen lo mismo: Python lee el archivo de arriba a abajo y ejecuta cada instrucción.

Forma	Cómo	Cuándo usarla
1. Botón del IDE ☆	**Spyder**: ▶ **VS-Code**: ▷ *Run Python File*	En el día a día
2. Terminal	`python nombre_archivo.py`	Cuando no hay IDE, o para ejecutar en otro computador

3. Consola	Después de ejecutar, escribir la llamada:
	
	`python -i nombre_archivo.py`
	`>>> nombre_funcion(argumentos)`
		Para probar una función sola




```
Programa de conversion de °C a °F y K
--------------------------------
1. °C a °F
2. °C a K
3. Salir

Elija una opcion: 1
Digite la temperatura en °C: 100
100.0°C = 212.00°F

--------------------------------
1. °C a °F
2. °C a K
3. Salir

Elija una opcion: 3
Gracias por usarnos
```

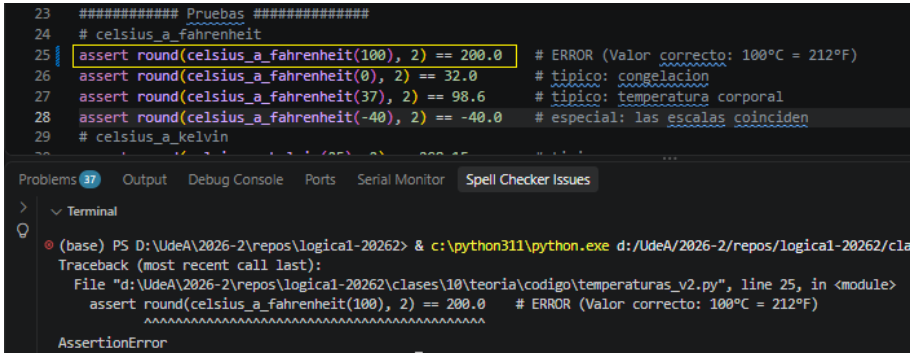
Ejecutar no es lo mismo que probar

- **Ejecutar:** correr el programa completo, como lo usaría el usuario.
- **Probar:** verificar que cada función da el resultado esperado.

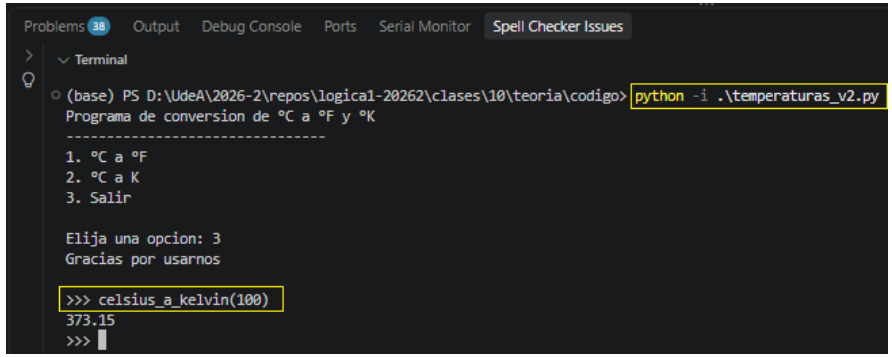
Si el archivo tiene pruebas con **`assert`**, cada ejecución también las corre:

Si las pruebas…	El programa…
pasan	sigue sin mostrar nada y arranca normalmente
fallan	se detiene con **`AssertionError`** e indica la línea que falló

Desde la consola se puede probar una función a mano, sin pasar por el resto del programa.



Una prueba falla. `AssertionError`



Probar función individualmente. `python -i`

Especificación de una función

- **¿Cómo alguien sabe como usar una función que no escribió?** Leyendo su especificación.
- La especificación es un **contrato** entre dos personas:

	Quién	**Compromiso**
Supone	quien **usa** la función	Entregar datos válidos
Garantiza	quien **escribe** la función	Devolver el resultado correcto, si los datos son válidos

En Python la especificación se escribe como un **docstring**, un texto entre `"""` justo debajo del **`def`**:

.

```
def celsius_a_kelvin(temp_c):
    """
    Convierte una temperatura de grados Celsius a Kelvin.

    Supone:
        temp_c es un número (int o float) mayor o igual a -273.15
        (el cero absoluto).
    Garantiza:
        Retorna la temperatura equivalente en Kelvin (float),
        mayor o igual a 0.
    """
    temp_k = temp_c + 273.15
    return temp_k
```

Especificación de una función

Idea clave: Con la especificación, una persona puede usar una función sin leer su código. Así varias personas pueden construir un mismo programa.

1. Para consultar como usar una función se emplea: **`help()`**

```
>>> help(celsius_a_kelvin)
Help on function celsius_a_kelvin in module __main__:

celsius_a_kelvin(temp_c)
    Convierte una temperatura de grados Celsius a Kelvin.

    Supone:
        temp_c es un número (int o float) mayor o igual a -273.15
        (el cero absoluto).
    Garantiza:
        Retorna la temperatura equivalente en Kelvin (float),
        mayor o igual a 0.
>>>
```

2. El programa no da error, pero el resultado **no tiene sentido**: no existen temperaturas en Kelvin negativas.

```
Elija una opcion: 2
Digite la temperatura en °C: -300
-300.0°C = -26.85K
```

¿De quién es la culpa? De quien llamó a la función con -300, porque no cumplió lo que la función supone. La función cumplió su parte.

Ámbito de las variables

Ámbito: la parte del programa donde una variable existe y se puede usar.

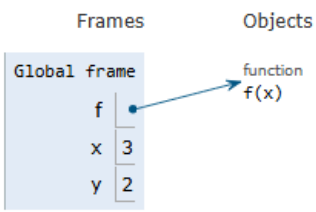
- **Variable local**: se crea dentro de una función, y solo existe mientras la función se ejecuta. Los parámetros también son locales.
- **Variable global**: se crea fuera de las funciones, en el programa principal.

Código

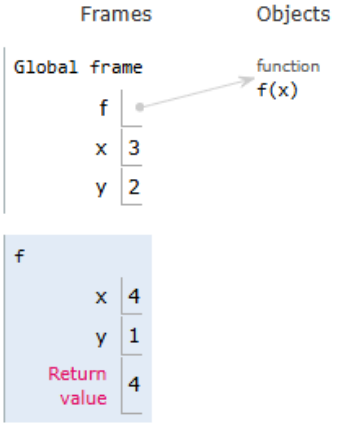
```
1  def f(x):
2      y = 1
3      x = x + y
4      print('x =', x)
5      return x
6
7  x = 3
8  y = 2
9  z = f(x)
10 print('z =', z)
11 print('x =', x)
11 print('y =', y)
```

Link simulación

→ 9 `z = f(x)`

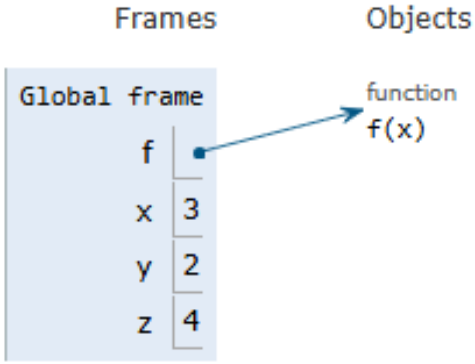


→ 5 `return x`



Print output

```
x = 4
z = 4
x = 3
y = 2
```



Salida en pantalla

```
x = 4      ← la x local de f
z = 4
x = 3      ← la x global no cambió
y = 2      ← la y global no cambió
```

Ámbito de las variables

- **Idea clave**: dos variables pueden tener el mismo nombre y ser distintas, si viven en ámbitos diferentes. Cada llamada a una función crea su propio espacio de variables.

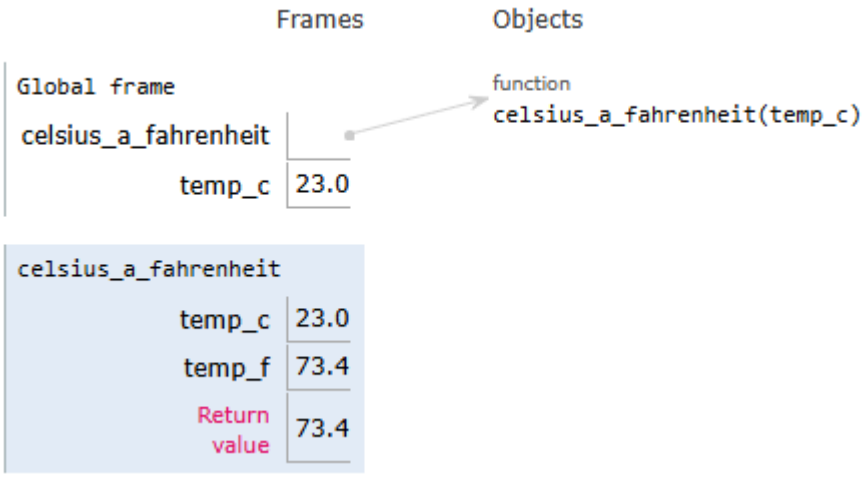
Código

```
1  def celsius_a_fahrenheit(temp_c):
2      temp_f = (9 / 5) * temp_c + 32
3      return temp_f
4
5  temp_c = float(input("Ingrese los °C: "))
6  temp_f = celsius_a_fahrenheit(temp_c)
7
8  print(f"{temp_c} °C = {temp_f} °F")
```

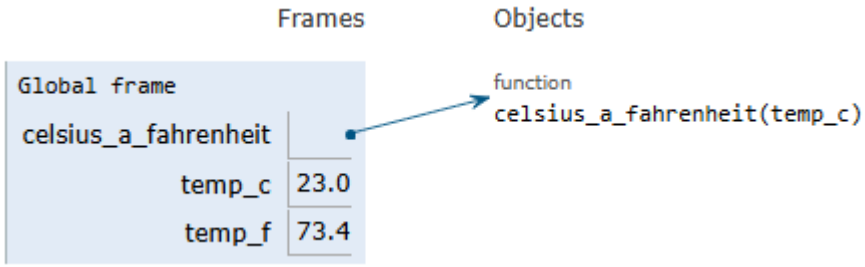
Salida en pantalla

```
Ingrese los °C: 23
23.0 °C = 73.4 °F
```

Link simulación



→ 3 `return temp_f`



→ 9 `Fin del programa`

Variables globales: ¿Sí o No?

Idea clave: una función puede leer una variable global, pero no debería modificarla. Lo que necesita lo recibe por parámetros y lo que produce lo entrega con return.

Si: **constantes globales** (en MAYÚSCULA; su valor nunca cambia)

```
CERO_ABSOLUTO = -273.15

def celsius_a_kelvin(temp_c):
    temp_k = temp_c - CERO_ABSOLUTO
    return temp_k

temp_k = celsius_a_kelvin(25)
print("25 °C =", temp_k, "K")
```

Link simulación

Salida en pantalla

```
25 °C = 298.15 K
```

No: modificar una global con `global`

```
conversiones = 0

def celsius_a_fahrenheit(temp_c):
    global conversiones
    conversiones = conversiones + 1
    temp_f = (9 / 5) * temp_c + 32
    return temp_f

# Pruebas
assert round(celsius_a_fahrenheit(100), 2) == 212.0
assert round(celsius_a_fahrenheit(0), 2) == 32.0

# Programa principal: el usuario hace UNA conversion
temp_f = celsius_a_fahrenheit(23)
print("23 °C =", temp_f, "°F")
print("Conversiones:", conversiones)
```

Link simulación

Salida en pantalla

```
23 °C = 73.4 °F
Conversiones: 3
```

Errores típicos de ámbito

Si no se respeta el ámbito de las variables, Python lo avisa con un **error**.

Caso 1: usar afuera una variable local.

```
def celsius_a_fahrenheit(temp_c):
    temp_f = (9 / 5) * temp_c + 32
    return temp_f

# el resultado no se guarda
celsius_a_fahrenheit(23)
print(temp_f)
```

Link simulación

Salida en pantalla

```
NameError: name 'temp_f' is not defined
```

Solución

Guardar el resultado: `temp_f = celsius_a_fahrenheit(23)`

Caso 2: modificar una global sin global.

```
conversiones = 0

def celsius_a_fahrenheit(temp_c):
    conversiones = conversiones + 1
    temp_f = (9 / 5) * temp_c + 32
    return temp_f

temp_f = celsius_a_fahrenheit(23)
```

Link simulación

Salida en pantalla

```
UnboundLocalError: cannot access local
variable 'conversiones' where it is not
associated with a value
```

Recomendación

Contar en el programa principal, no dentro de la función.

Las variables globales hacen que los programas sean difíciles de entender y depurar.

Formas de pasar argumentos

Python a diferencia de otros lenguajes de programación (como C por ejemplo), permite dos formas de pasar argumentos:

- **Por posición**: los argumentos se asignan en el mismo orden de los parámetros.
- **Por nombre**: se escribe **`parametro = valor`**, y el orden no importa.

$$\text{sen}(x) = 1 - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots$$

```
def factorial(n):
    fact = 1
    for i in range(1, n + 1):
        fact = fact * i
    return fact

def seno(x, n):
    suma = 0
    for k in range(n):
        suma = suma + (-1)**k * x**(2*k + 1) / factorial(2*k + 1)
    return suma

a = seno(1.0, 3)          # por posicion
b = seno(x=1.0, n=3)      # por nombre
c = seno(n=3, x=1.0)      # por nombre, en otro orden
d = seno(1.0, n=3)        # mezclados
# e = seno(n=3, 1.0)      # SyntaxError
print(a, b, c, d)
```

Link simulación

Regla: se pueden mezclar, pero los argumentos por nombre van después de los posicionales. Si no, Python no ejecuta ninguna línea: **`SyntaxError: positional argument follows keyword argument.`**

Parámetros con valor por defecto

Problema: con la versión anterior, si se omite **n**:

```
>>> seno(1.0)
TypeError: seno() missing 1 required positional argument: 'n'
```

Solución: Darle a **n** un valor por defecto:

```
def seno(x, n=10):                  # parámetro con valor por defecto
    suma = 0
    for k in range(n):
        suma = suma + (-1)**k * x**(2*k + 1) / factorial(2*k + 1)
    return suma

a = seno(1.0)           # ← argumento omitido: usa n = 10
b = seno(1.0, 3)        # n = 3
c = seno(n=3, x=1.0)    # ← argumentos asignados por nombre
```

Link simulación

Salida en pantalla:

```
0.8414709848078965      ← 10 términos: coincide con el valor real
0.8416666666666667      ← 3 términos: error en la 4.ª cifra
0.8416666666666667
```

Regla: los parámetros con valor por defecto van al final. `def seno(n=10, x)`: da **`SyntaxError: parameter without a default follows parameter with a default`**.

Una función que llama a otra

- Una función puede usar otras funciones.
- Esto permite construir programas grandes a partir de piezas pequeñas y ya probadas.

```
 1  def factorial(n):
 2      fact = 1
 3      for i in range(1, n + 1):
 4          fact = fact * i
 5      return fact
 6
 7  def seno(x, n=10):
 8      suma = 0
 9      for k in range(n):
10          suma = suma + (-1)**k * x**(2*k + 1) / factorial(2*k + 1)
11      return suma
12
13  resultado = seno(1.0, 3)
14  print(resultado)
```

Link simulación

Lo anterior permite:

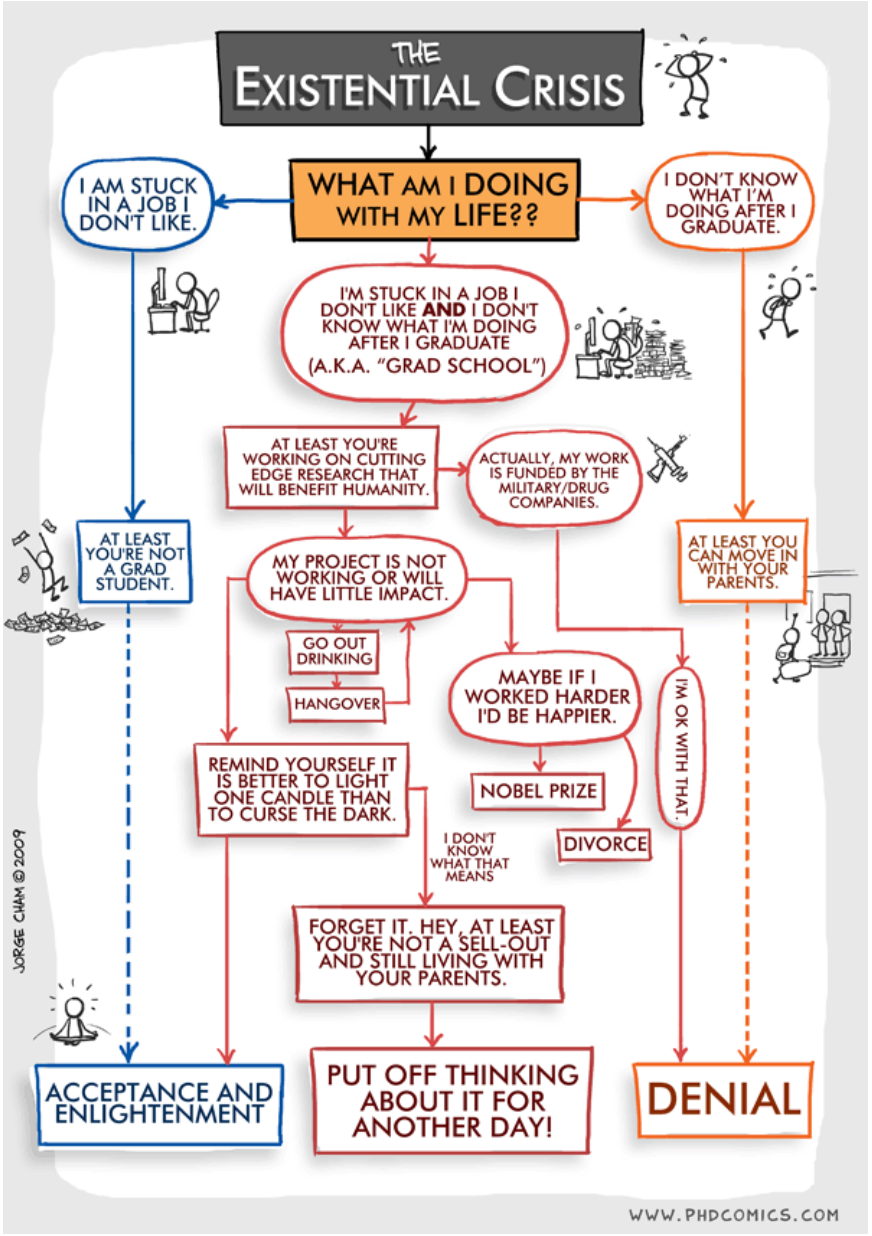
- **Reutilización**: **`factorial`** se escribe y se prueba una vez, y se usa donde se necesite.
- **Abstracción**: **`seno`** solo necesita saber **qué** hace **`factorial`**, no **cómo** lo hace.
- **La ejecución salta y vuelve**: al terminar **`factorial`**, se continúa en el punto exacto de la llamada (línea 10).
- **Pila de llamadas**: cada llamada crea su propio marco, con sus propias variables. Hay **dos n distintas**.

Referencias

1. https://ellibrodepython.com/
2. https://www.w3resource.com/index.php
3. https://realpython.com/
4. https://curriculumresources.edu.gh/wp-content/uploads/2025/01/
5. https://python-course.eu/
6. https://realpython.com/cheatsheets/python/
7. https://realpython.com/defining-your-own-python-function/
8. https://realpython.com/python-built-in-functions/
9. https://realpython.com/ref/best-practices/functions/
10. https://makecode.microbit.org/courses/csintro
11. https://makecode.microbit.org/courses
12. https://realpython.com/python-while-loop/
13. https://realpython.com/ref/keywords/while/
14. https://cs.stanford.edu/people/nick/py/

Eso es todo amigos





https://phdcomics.com/comics/archive.php?comicid=1431